MongoDB Interview Questions and Answers (Detailed & Comprehensive)

---

1. **Basic-Level MongoDB Questions**

1.1 What is MongoDB?

MongoDB is a **NoSQL**, **document-oriented**, **schema-less** database that stores data in **BSON (Binary JSON)** documents. It provides high performance, horizontal scalability, and flexible schema design.

1.2 What is a document in MongoDB?

A document is the fundamental data unit represented in BSON. Example:

```
{
  "name": "Dushyant",
  "age": 28,
  "skills": ["Node.js", "React"],
  "address": { "city": "Mumbai", "zip": 400001 }
}
```

1.3 What is a collection?

A collection is a grouping of documents, similar to a table in RDBMS.

1.4 Explain the difference between MongoDB and MySQL.

Factor	MongoDB	MySQL
Model	Document-based	Relational
Schema	Schema-less	Fixed schema
Joins	Limited (lookup)	Full joins
Scaling	Horizontal (sharding)	Vertical
Transactions	Supported (multi-doc since v4.0)	Fully ACID

1.5 What is BSON?

BSON is “Binary JSON.” It supports additional data types beyond JSON, such as:

- Date
- Binary data
- ObjectId
- Decimal128

1.6 What is an ObjectId?

A 12-byte unique identifier automatically generated by MongoDB.

Structure:

- 4-byte timestamp
- 5-byte random value
- 3-byte incrementing counter

1.7 How do you insert a document in MongoDB?

```
db.users.insertOne({ name: "Alice", age: 25 });
```

1.8 How do you query documents?

```
db.users.find({ age: { $gte: 25 } })
```

1.9 What is a capped collection?

A fixed-size collection that overwrites old documents when full. Supports high-speed inserts.

1.10 Does MongoDB support transactions?

Yes. From MongoDB 4.0 onward, multi-document ACID transactions are supported in replica sets; from 4.2 in sharded clusters.

---

2. **CRUD and Query Operators**

2.1 Explain the difference between find(), findOne(), and findOneAndUpdate().

- **find()**: Returns a cursor of documents.
- **findOne()**: Returns a single document.
- **findOneAndUpdate()**: Atomically finds, updates, and returns the modified or original document.

2.2 What are update operators?

Examples:

- `$set`: Modify specific fields.
- `$unset`: Remove fields.
- `$inc`: Increment values.
- `$push`: Add to array.
- `$addToSet`: Add unique array element.

Example:

```
db.users.updateOne({ name: "Dushyant" }, { $set: { age: 29 } });
```

2.3 What is the difference between deleteOne() and deleteMany()?

- **deleteOne()** removes the first matched document.
- **deleteMany()** removes all matched documents.

2.4 What is a projection?

Projection determines which fields to include/exclude.

```
db.users.find({}, { name: 1, age: 1, _id: 0 })
```

3. **Indexes**

3.1 What is an index in MongoDB?

Indexes improve query performance by allowing MongoDB to avoid scanning all documents.

3.2 Types of indexes

1. Single-field index
2. Compound index
3. Multikey index (for arrays)
4. Text index
5. Geospatial index
6. Hashed index
7. Wildcard index

3.3 How to create an index?

```
db.users.createIndex({ age: 1 });
```

3.4 What is a compound index?

Index on multiple fields.

```
db.users.createIndex({ age: 1, name: -1 });
```

3.5 What is index cardinality?

Cardinality refers to the number of unique values in an index. Higher cardinality = better performance.

3.6 Covered queries

Queries satisfied entirely by the index without reading documents.

Requirements:

- Query fields included in index
- Projection fields included in index
- Exclude `_id` unless indexed

3.7 How to view indexes?

```
db.users.getIndexes();
```

3.8 How does MongoDB choose indexes?

Uses the **query planner** to evaluate multiple candidate plans and selects the most efficient one.

---

4. **Aggregation Framework**

4.1 What is aggregation?

Aggregation processes data using stages, similar to pipelines.

4.2 Common aggregation stages:

- `$match`
- `$group`
- `$project`
- `$sort`
- `$limit`
- `$lookup`
- `$unwind`
- `$addFields`

4.3 Example aggregation pipeline:

```
db.orders.aggregate([
  { $match: { status: "completed" } },
  { $group: { _id: "$customerId", totalSpent: { $sum: "$amount" } } },
  { $sort: { totalSpent: -1 } }
]);
```

4.4 What is $lookup?

Performs a left outer join.

```
{
  $lookup: {
    from: "orders",
    localField: "_id",
    foreignField: "userId",
    as: "orders"
  }
}
```

---

5. **Schema Design**

5.1 What is embedding vs referencing?

Embedding (denormalization)

- Faster reads
- Best for 1:1 or 1:few relationships

Referencing (normalization)

- Better for large 1:many or many:many
- Avoids document growth

5.2 What is document growth?

MongoDB documents grow when new fields are added. If the allocated record size is exceeded, MongoDB relocates the record → affects performance.

5.3 When to embed?

- Data accessed together
- Atomicity needed
- Low cardinality

5.4 When to reference?

- Large arrays
- High-write workloads
- Many-to-many relationships

---

6. **Replication**

6.1 What is replication?

Replication ensures:

- Data redundancy
- High availability
- Failover support

6.2 Components of a replica set

- Primary
- Secondary
- Arbiter (optional)

6.3 What is replica set failover?

If the primary fails:

- Secondaries initiate an election
- New primary is chosen

6.4 Types of replication nodes

- Primary
- Secondary
- Hidden secondary
- Delayed secondary
- Arbiter

6.5 Write concerns

Controls the acknowledgment of write operations.

- `w: 1` (default)
- `w: majority`
- `w: 0` (unacknowledged)

6.6 Read concerns

- `local`
- `majority`
- `available`
- `linearizable`
- `snapshot`

---

7. **Sharding**

7.1 What is sharding?

Horizontal scaling technique that distributes data across multiple machines.

7.2 Components of sharded clusters

- **Shard servers** (store data)
- **Config servers** (metadata)
- **Mongos** (query router)

7.3 Shard key requirements

- High cardinality
- Good distribution
- Low frequency of updates
- Monotonic keys cause chunk hotspot

7.4 What is chunk?

A chunk is a 64MB logical partition of data. MongoDB auto-balances chunks across shards.

---

8. **Transactions**

8.1 What is a multi-document transaction?

Allows ACID compliance across multiple documents.

8.2 Example in MongoDB:

```
session = db.startSession();
session.startTransaction();

try {
  session.getDatabase("shop").orders.insertOne({ orderId: 1 });
  session.getDatabase("shop").inventory.updateOne({ item: "pen" }, { $inc: { qty:
-1 } });
  session.commitTransaction();
} catch(e) {
  session.abortTransaction();
}
```

---

9. **MongoDB Internals**

9.1 WiredTiger storage engine features:

- Document-level locking
- Compression (Snappy, Zlib)
- Checkpointing
- High concurrency

9.2 Journaling

Ensures crash recovery.

9.3 oplog

Replication log that stores operations applied to primary.

9.4 Write Amplification

Due to journaling + checkpoints. Can be reduced via compression.

---

10. **Security**

10.1 Authentication mechanisms

- SCRAM-SHA-1/SCRAM-SHA-256
- x.509 certs
- LDAP
- Kerberos

10.2 What is role-based access control (RBAC)?

Assigns users roles with permissions.

10.3 Enable access control

```
mongod --auth
```

10.4 Encryption

- In-transit: TLS
- At-rest: WiredTiger encryption

---

11. **Performance Tuning**

11.1 Query performance tools

- `explain()`
- `profiler`
- `mongostat`
- `mongotop`

11.2 Common performance issues

- Missing indexes
- Large documents (>16MB)
- Unbounded arrays
- Slow aggregations
- Hot shard key

11.3 How to fix slow queries?

- Add indexes
- Rewrite query
- Avoid full collection scans
- Use projections
- Use covered queries

---

12. **Admin/DevOps Questions**

12.1 How to take backup?

- `mongodump`
- File system snapshots
- Cloud backup

12.2 How to restore backup?

```
mongorestore
```

12.3 Monitoring tools

- Cloud Manager
- Ops Manager
- MongoDB Atlas metrics

---

13. **Advanced Questions**

13.1 What is snapshot isolation?

Ensures read operations see a consistent snapshot of data.

13.2 Explain internal architecture of aggregation pipeline.

- Pipeline is split across stages
- Mongos distributes work in sharded cluster
- Uses memory and disk for large sorts

13.3 What are change streams?

Real-time data changes notification. Example:

```
db.collection.watch();
```

13.4 What is the difference between:

Aggregation framework vs MapReduce?

Aggregation	MapReduce
Faster	Slower
Pipeline-based	JS-based
Native engine	Deprecated

13.5 What is write throttling?

Occurs when the storage engine becomes saturated.

---

14. **Practical Coding Questions**

Q: Find users with skill “Node.js”

```
db.users.find({ skills: "Node.js" });
```

Q: Update nested field inside array

```
db.users.updateOne(
  { "skills.name": "JavaScript" },
  { $set: { "skills.$.experience": 5 } }
)
```

Q: Pagination

```
db.users.find().skip(20).limit(10);
```

Q: Unique index

```
db.users.createIndex({ email: 1 }, { unique: true });
```

---

15. **HR + Scenario-Based MongoDB Questions**

Q: Design a schema for an eCommerce system.

Key points:

- Embed order items
- Reference customers
- Use compound indexes: `{ userId, orderDate }`
- Shard key = `userId` or `region`

Q: How do you handle a hot shard?

- Choose better shard key
- Use hashed shard keys
- Pre-split chunks

Q: How do you migrate large collections?

- Use mongodump/mongorestore
- Use chunk-level migration
- Use aggregation with `$out` or `$merge`
